

Javascript Function

| Content

Params	Callback	Scope	Closure
Context	Change Context	Arrow Function	Render

Default Parameter

JavaScript does not support **function overloading** in the traditional way like Java or C++.

JavaScript uses a mechanism called **hoisting**, where function declarations are moved to the top of their scope. If you define multiple functions with the same name, the later definition will override the earlier one.

```
function multiply(a, b) {  
    return a * b  
}
```

```
multiply(5, 2) // 10  
multiply(5) // NaN
```

```
function multiply(a, b) {  
    b = (b === undefined) ? b : 1;  
    return a * b  
}
```

```
multiply(5) // 5
```

```
function multiply(a, b = 1) {  
    return a * b;  
}
```

```
multiply(5) // 5
```

```
multiply(5, undefined) // 5
```

```
multiply(5, null) // 0
```

Careful with object parameter

```
const myCar = {
  userName: "Honda",
  color: "black"
}
function decorate(car){
  car.color = "red";
}
decorate(myCar);
console.log(myCar);
// { userName: 'Honda', color:
  'red' }
```

Arguments

```
function sum(a, b, c) {  
    console.log(arguments)  
    return a + b + c;  
}
```

```
sum(1, 2, 3);  
// [Arguments]{ '0': 1, '1': 2, '2': 3 }  
// have length, no array method
```

```
function sum() {  
    const len = arguments.length;  
    let total = 0;  
    for (let i = 0; i < len; i++) {  
        total += arguments[i];  
    }  
    console.log(total);  
    return total;  
}
```

```
sum(1, 2, 3); // 6  
sum(1, 2, 3, 4, 5); // 15
```

Rest parameter

You cannot define multiple rest parameters in a function. Each function can only have **one** rest parameter, and it must be **the last** parameter in the function's parameter list.

```
function sum() {
  const len = arguments.length;
  let total = 0;
  for (let i = 0; i < len; i++) {
    total += arguments[i];
  }
  console.log(total);
  return total;
}
sum(1, 2, 3); // 6
sum(1, 2, 3, 4, 5); // 15
```

```
function sum(...numbers) {
  let total = 0;
  numbers.forEach(num => total += num);
  console.log(total);
  return total;
}
function sum(...numbers) {
  return numbers.reduce((total, num) => total + num);
}
console.log(sum(1, 2, 3, 4, 5, 6));
```

Callback

```
function loadImg(src, callback) {  
    const img = new Image();  
    img.onload = function () {  
        callback && callback(img);  
    };  
    img.src = src;  
}
```

```
function addImage(img) {  
    document.body.appendChild(img);  
    img.width = img.naturalWidth * 2;  
    img.height = img.naturalHeight * 2;  
}  
loadImg("./background.jpg", addImage);
```

Callback Hell

```
1  function hell(win) {
2    // for listener purpose
3    return function() {
4      loadLink(win, REMOTE_SRC+'/assets/css/style.css', function() {
5        loadLink(win, REMOTE_SRC+'/lib/async.js', function() {
6          loadLink(win, REMOTE_SRC+'/lib/easyXDM.js', function() {
7            loadLink(win, REMOTE_SRC+'/lib/json2.js', function() {
8              loadLink(win, REMOTE_SRC+'/lib/underscore.min.js', function() {
9                loadLink(win, REMOTE_SRC+'/lib/backbone.min.js', function() {
10                 loadLink(win, REMOTE_SRC+'/dev/base_dev.js', function() {
11                  loadLink(win, REMOTE_SRC+'/assets/js/deps.js', function() {
12                   loadLink(win, REMOTE_SRC+'/src/' + win.loader_path + '/loader.js', function() {
13                    async.eachSeries(SRIPTS, function(src, callback) {
14                     loadScript(win, BASE_URL+src, callback);
15                    });
16                   });
17                  });
18                 });
19                });
20               });
21              });
22             });
23            });
24           });
25          });
26         }
```



Global Scope

Variables and functions declared in the global scope are called **global variables** and **global functions**.

```
// common.js
const config = {
  designResolution: {
    width: 1280,
    height: 720,
  },
  orientation: "landscape"
}
module.export = config;
// loadImage.js
const { designResolution } = config;
console.log(designResolution);
// {width: 1280, height: 720}
```

```
// index.js
const loadImage = require("../src/loadImage.js");
//output { width: 1280, height: 720 }
```

Variables not inside any function is
global scope

Local Scope

Variables and functions declared in the local scope can only be accessed from within that function or code block.

Variables inside function or block is local scope

```
function loadImg(src, callback) {  
  const img = new Image();  
  // local scope 1  
  
  img.onload = function () {  
    // local scope 2  
    callback && callback(img);  
  };  
  img.src = src;  
}
```

```
function loadImg(src, callback) {  
  const img = new Image();  
  console.log(scale); // error  
  img.onload = function () {  
    let scale = 2;  
    callback && callback(img);  
    img.width = img.naturalWidth * scale;  
    img.height = img.naturalHeight * scale;  
    console.log(img); // img  
  };  
  img.src = src;  
}
```

Child scope including parent scope (lexical scope)

Block Scope

Block Scope is the scope inside a code block enclosed in curly braces {}.

Write the function countdown time $n(s) \rightarrow 0$

```
function countdown(time) {  
  for (var i = time; i >= 0; i--) {  
    setTimeout(function () {  
      console.log(i);  
    }, 1000 * (time - i))  
  }  
}  
countdown(3);  
// -1 ... -1 ... -1
```

```
function countdown(time) {  
  for (let i = time; i >= 0; i--) {  
    setTimeout(function () {  
      console.log(i);  
    }, 1000 * (time - i))  
  }  
}  
countdown(3);  
// 3 ... 2 ... 1
```

let, const only work inside {}

var work inside the function

Scope

How they handle that before `let`, `const`

```
function countdown(time) {
  for (var i = time; i >= 0; i--) {
    (function(n){
      setTimeout(function () {
        console.log(n);
      }, 1000 * (time - i))
    })(i);
  }
}
```

```
function countdown(time) {
  for (var i = time; i >= 0; i--) {
    function delay(n){
      setTimeout(function () {
        console.log(n);
      }, 1000 * (time - i))
    }
    delay(i);
  }
}
```

Scope Summary

```
// Global Scope
let globalVar = "I am a global variable";

function outerFunction() {
  // Outer Function Scope
  let outerVar = "I am an outer variable";

  function innerFunction() {
    // Inner Function Scope
    let innerVar = "I am an inner variable";
    console.log(innerVar); // Access local variable
    console.log(outerVar); // Access variable from outer scope
    console.log(globalVar); // Access global variable
  }

  innerFunction();
  console.log(innerVar); // Error: innerVar is not defined here
}

outerFunction();
console.log(innerVar); // Error: innerVar is not defined here
```

Scope Summary

Differences between **Local Scope** and **Block Scope**.

Key differences:

- Local Scope applies to variables and functions declared inside a **function**, while Block Scope applies to variables declared with **let** or **const** inside a **code block**.
- Local Scope does not depend on the use of **let** or **const**, while Block Scope only applies to variables declared with **let** or **const**.

Note:

Variables declared with **var** do not have block scope; they have **function scope**, meaning they can be accessed from outside the code block if they are declared inside a function.

Closures

Closures is all accessible variables when function created

```
const { designResolution, isResize } = config;
function addImage(img) {
  const { naturalWidth, naturalHeight } = img;
  function getScale() {
    const { width, height } = designResolution;
    return Math.min(naturalWidth / width, naturalHeight / height);
  }
  if (isResize) {
    const scale = getScale();
    return function () {
      // closure
      img.width = naturalWidth * scale;
      img.height = naturalHeight * scale;
    }
  }
}
```

Closures

Closures is all accessible variables when function created

- own scope
- parent scope
- global scope
- the arguments of the outer function
- the function has returned.

Context

What is `this`?

In an object method	<code>this</code> refers to the object.
Alone	<code>this</code> refers to the global object
In a function	<code>this</code> refers to the global object
In a function, in strict mode	<code>this</code> is undefined.
In an event	<code>this</code> refers to the element that received the event.

Context

What is **this**?

```
console.log(this);  
// window  
function log() {  
  console.log(this);  
  // "use strict" ? undefined : window  
}  
const dog = {  
  bark: function(){  
    console.log("gau gau", this);  
    // gau gau {bark: f}  
  }  
}
```

```
window.onload = function () {  
  const button = document.getElementById("Btn");  
  button.addEventListener("click", dog.bark);  
}  
// gau gau <button id="BtnClick">Click Me</button>
```

call, apply

- **Call and Apply:** Both call the function **immediately** with the new context. `call` takes individual arguments, while `apply` takes an array of arguments.

```
const dog = {
  sound: "gaugau",
  makeSound: function () {
    console.log(this.sound);
  }
}
const cat = {
  sound: "meomeo",
}
dog.makeSound.call(cat);
// meomeo
dog.makeSound.apply(cat);
// meomeo
```

```
const dog = {
  sound: "gaugau",
  makeSound: function (emo1, emo2) {
    console.log(this.sound, emo1, emo2);
  }
}
const cat = {
  sound: "meomeo",
}
dog.makeSound.call(cat, "hungry", "hungry");
// meomeo hungry hungry
dog.makeSound.apply(cat, ["hungry", "gruwww"]);
// meomeo hungry gruwww
```

bind

- **Bind:** **Creates** a new function with a bound context but does **not** call the function **immediately**. This new function can be called later.

```
const dog = {
  sound: "gaugau",
  makeSound: function (emo1, emo2) {
    console.log(this.sound, emo1, emo2);
  }
}
const cat = {
  sound: "meomeo",
}
cat.makeSound = dog.makeSound.bind(cat);
cat.makeSound("gruw", "gruw");
// meomeo gruw gruw
```

| arrow function

```
const max = function(a, b){  
    return a > b ? a : b;  
}  
const max = (a, b) => a > b ? a : b;
```

- Do not have own this
- Do not have arguments
- Can not using with call, bind, apply
- Can not using as constructor
- Should not be used as object methods
- **this** is the current **this** when function created

```

const user = {
  name: "Ronaldo",

  // Regular function
  sayNameRegular: function() {
    console.log(`Regular: ${this.name}`);

    // this points to global object (Window)
    setTimeout(function() {
      console.log(`Regular timeout: ${this.name}`);
    }, 1000);
  },
};

user.sayNameRegular();
// Regular: Ronaldo
// Regular timeout: undefined (lost this context)

```

```

const user = {
  name: "Ronaldo",

  // Arrow function
  sayNameArrow: function() {
    console.log(`Arrow: ${this.name}`);

    // this inherits from outer context (user)
    setTimeout(() => {
      console.log(`Arrow timeout: ${this.name}`);
    }, 1000);
  },
};

user.sayNameArrow();
// Arrow: Ronaldo
// Arrow timeout: Ronaldo (preserved this context)

```

Assignment 1

Making a clock

- Create a clock like that image



Assignment 2

The pair game

- There are 20 cards with 10 different images.
- open 2 cards each time,
- if they are matched, hide them, get 1000 coin.
- if they are not, close them, loss 500 coin.
- first coin is 10.000.
- if coin < 0 => game over.



| How to ask

1. What you want to do?
2. What did you do, and what is your issue?
3. Capture of your code or the error.

Homework